

The finite difference method for the viscous Burgers equation

Explicit finite differences on a periodic, validated against the exact solution (using Cole-Hopf transformation)

Ruben Ranval

ruben.ranval@polytechnique.edu

Introduction

The Burgers equation is a simple canonical one-dimensional model in which a convective nonlinearity competes with viscous diffusion. The reason, I'm using it is that it's the simplest equation that keeps the quadratic nonlinearity of Navier Stokes while remaining exactly solvable (which makes it an ideal benchmark for numerical methods).

In this notebook, I will solve the Burgers equation on a periodic domain with a fully explicit FD scheme, and then validate the result against the exact solution obtained through the Cole-Hopf transformation.

The Mathematical problem

Governing equation

We want to solve

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \nu \frac{\partial^2 u}{\partial x^2} \quad \forall x \in [0, 2\pi] \quad t \in [0, 2]$$

with periodic BCs and initial conditions $u(0, x) = \sin(x)$. The advection term is taken in conservative form, $u \frac{\partial u}{\partial x} = \frac{\partial}{\partial x} \left(\frac{u^2}{2} \right)$, which is the form used for the discretization below. The constant $\nu > 0$ is the kinematic viscosity.

Physical behavior

Two mechanisms compete: the nonlinear term that steepens the profile, and for small ν tends to form a shock, while diffusion smooths gradients and spreads the solution. Their balance is of course

measured by the Reynolds number. With $\nu = 0.5$ (the default value of this notebook) the run is strongly diffusive: the initial sine flattens and decays without forming a sharp front (as the results of the simulation will confirm).

Numerical scheme

Spatial discretization

Space is discretized on a uniform periodic grid of N_0 points with spacing $\Delta x = \frac{2\pi}{N_0}$. Writing u_j for the value at node j , both spatial terms use second - order central differences :

- for the advection term: $\frac{\partial}{\partial x} \left(\frac{u^2}{2} \right) \approx \frac{u_{j+1}^2 - u_{j-1}^2}{4 \Delta x} = a \Delta v$
- for the diffusion term: $\nu \frac{\partial^2 u}{\partial x^2} \approx \nu \frac{u_{j+1} - 2u_j + u_{j-1}}{\Delta x^2} = d \Delta f f$

Time integration and stability

Time advances with explicit (forward) Euler, giving the classic forward time central space update:

$$u_j^{n+1} = u_j^n - \Delta t \cdot a \Delta v + \Delta t \cdot d \Delta f f.$$

FTCS is only conditionally stable. A Von Neumann analysis of the linearized equation gives the following amplification factor: $|G(\theta)|^2 = (1 - 2d(1 - \cos(\theta)))^2 + (C \sin(\theta))^2$, with $d = \nu \frac{\Delta t}{\Delta x^2}$, and $C = a \frac{\Delta t}{\Delta x}$ with a the local wave speed.

Stability ($|G| \leq 1$ for every mode θ) requires both $2d \leq 1$ and $C^2 \leq 2d$. The step is therefore chosen as $\Delta t = \min \left(0.2 \frac{\Delta x^2}{\nu}, 0.3 \frac{\Delta x}{A} \right)$: the first term enforces the diffusion limit, the second the advective CFL limit (both with a safety factor).

The actual implementation!

Setup: parameters and grid

We fix the domain, the physical parameters and the grid. N_0 is entered as a real number (256.0), this forces every downstream quantity to machine-precision arithmetic and prevents an exact symbolic π from being dragged through the time loop, which would make the run explode in symbolic size.

```

L = 2 * Pi;    (*length of the domain, chose 2pi since it's a sine*)
N0 = 256.0;    (*nbr of grid points*)
dx =  $\frac{L}{N0}$ ;    (*grid spacing*)
v = 0.5;      (*kinematic viscosity*)
A = 1.0;      (*characteristic wave speed*)

```

Now the two CFL limits we discussed in the previous section (with their safety factors):

In[186]:=

```

dt = Min[ $0.2 * \frac{dx^2}{v}$ ,  $0.3 * \frac{dx}{A}$ ];

```

Now let's generate the timesteps and the initial conditions:

In[187]:=

```

tFinal = 2.0;
instants = Floor[ $\frac{tFinal}{dt}$ ];
xCoords = Table[i * dx, {i, 0, N0 - 1}];
initU = Sin@xCoords;

```

The time-stepping “kernel”

stepBurgers advances the whole field by one time step. It is written as a pure function of the state vector so it can be iterated cleanly. *uR* and *uL* represent the right/left neighbors via array rotation.

In[191]:=

```

stepBurgers[u_, v_, dt_, dx_] :=
Module[{uL, uR, adv, diff},
  uR = RotateLeft[u, 1];
  uL = RotateRight[u, 1];
  adv =  $\frac{uR^2 - uL^2}{4 dx}$ ;
  diff =  $v * \frac{(uR - 2 u + uL)}{dx^2}$ ;
  u - dt * adv + dt * diff
];

```

Time marching

We'll use *NestList* to build each time step iteratively using *stepBurgers* as the function we're applying, with *initU* as the initial conditions and *instants* the total number of timesteps.

In[192]:=

```

sol = NestList[stepBurgers[#, v, dt, dx] &, initU, instants];

```

The way we've structured our lists, we have *sol[[1, All]]* representing the solution at all 256 points of the 1D grid at the first instant, and *sol[[instants+1, All]]* representing the solution at all 256 points of the 1D grid at the last instant of the simulation.

```
In[225]:= sol[[1, All]] == initU
```

```
Out[225]= True
```

Likewise, `sol[[All, 42]]` represents the position of 42nd point of the grid at all timesteps.

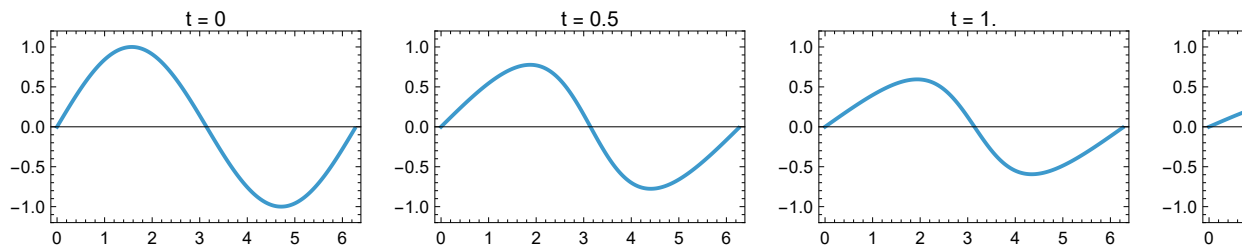
Now, let's look at the results!

Results

Snapshots of the numerical solution at $t = 0, 0.5, 1$ and 2 . The initial sine smooths and its amplitude decays monotonically: exactly what is expected in this diffusion dominated regime (here $v=0.5$).

```
In[234]:= GraphicsRow[
  Table[
    ListLinePlot[
      Transpose@{xCoords, sol[[Floor[t / dt] + 1]]}, PlotLabel -> "t = " <> ToString[t],
      PlotRange -> {-1.2, 1.2},
      Frame -> True],
    {t, {0, 0.5, 1.0, 2.0}}],
  ImageSize -> 800]
```

```
Out[234]=
```



The substitution $u = -2v \frac{\partial}{\partial x} (\log \phi)$ turns the nonlinear Burgers equation into the linear heat equation

$\frac{\partial \phi}{\partial t} = \frac{\partial^2 \phi}{\partial x^2}$. For $u(x, 0) = \sin(x)$ the matching heat-equation data is $\phi(x, 0) = \exp\left(\frac{\cos(x)}{2v}\right)$. Expanding

this with the generating function of the modified Bessel functions gives Fourier coefficients $I_n\left(\frac{1}{2v}\right)$, each mode then decays as $\exp(-n^2 v t)$. Reconstructing u from ϕ yields the reference solution below.

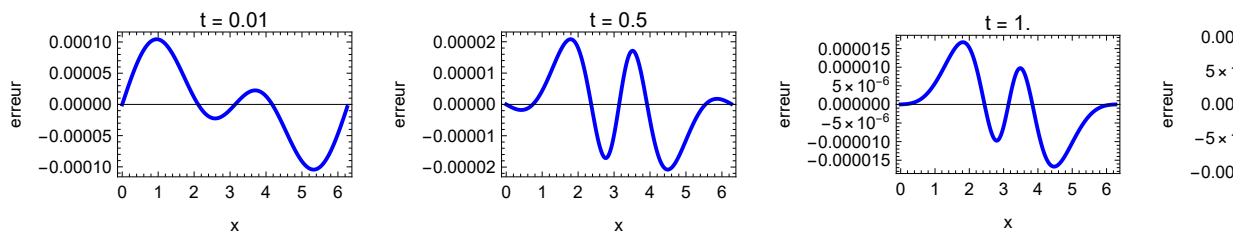
In[236]:=

```
solExacteBurgers[x_, t_, v_, nMax_: 20] :=
Module[{ϕ, dϕ, a},
  a[n_] := BesselI[n,  $\frac{1}{2\sqrt{v}}$ ];
  ϕ = Sum[a[n] * Exp[-n2 * v * t] * Exp[I * n * x], {n, -nMax, nMax}];
  dϕ = Sum[I * n * a[n] * Exp[-n2 * v * t] * Exp[I * n * x], {n, -nMax, nMax}];
  Re[-(2 * v *  $\frac{d\phi}{\phi}$ )]
];
```

Subtracting the exact solution from the numerical one gives the pointwise error. It stays at the order of and below across the domain and shrinks in time as the solution smooths.

```
GraphicsRow[
Table[
ListLinePlot[
Transpose@
{xCoords, sol[Floor[t / dt] + 1] - Table[solExacteBurgers[x, t, v], {x, xCoords}]},
...], {t, {0.01, 0.5, 1.0, 2.0}}, ImageSize -> 800]
```

Out[237]=



The exact solution is an infinite series truncated at nMax modes. The plot (shown at $v = 0.4$) compares nMax = 1, 2, 3, 10, 20. A single mode is hopeless at early times, when the profile is steepest and richest in high harmonics, but the series converges fast: a handful of modes already coincide visually, and fewer are needed at later times once diffusion has erased the fine structure.

```
GraphicsRow[
  Table[Plot[Evaluate@Table[solExacteBurgers[x, t, 0.4, n], {n, {1, 2, 3, 10, 20}}],
    {x, 0, 2 Pi}, {t, {0.5, 2.0}}, ImageSize -> 900]
```

Out[239]=

